Machine Learning & Data Mining
Set 3

Caltech CS/CNS/EE 155
October 13th, 2026

Policies

- Due 11:59 PM PT, October 20th, 2026, via Gradescope.
- You are free to collaborate on all of the problems, subject to the collaboration policy stated in the syllabus.
- In this course, we will be using Google Colab for code submissions. You will need a Google account.
- Homework will be graded on completion, but you must make a good-faith effort. No late submissions will be accepted.
- Students are expected to complete homework assignments based on their understanding of the course material. Student can use LLMs as a resource (e.g., helping with debugging, or grammar checking), but the assignments (including code) should be principally authored by the student.

Submission Instructions

- Submit your report as a single .pdf file to Gradescope (entry code PRGP5Z), under "Problem Set 3".
- In the report, **include any images generated by your code** along with your answers to the questions.
- Submit your code by **sharing a link in your report** to your Google Colab notebook for each problem (see naming instructions below). Make sure to set sharing permissions to at least "Anyone with the link can view".
- For instructions specifically pertaining to the Gradescope submission process, see https://www.gradescope.com/get_started#student-submission.

Google Colab Instructions

For each notebook, you need to save a copy to your drive.

1. Open the github preview of the notebook, and click the icon to open the colab preview.
2. On the colab preview, go to File → Save a copy in Drive.
3. Edit your file name to "lastname_firstname_set_problem", e.g."yue_yisong_set3_prob1.ipynb"

1 Decision Trees [30 Points]

Relevant materials: Lecture 5

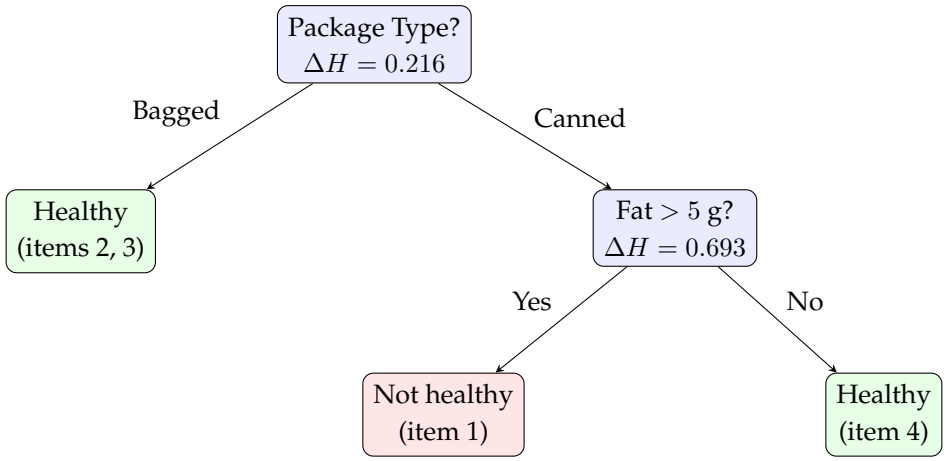
Problem A [7 points]: Consider the following data, where given information about some food you must predict whether it is healthy:

No.	Package Type	Unit Price $>$ \$5	Contains $>$ 5 grams of fat	Healthy?
1	Canned	Yes	Yes	No
2	Bagged	Yes	No	Yes
3	Bagged	No	Yes	Yes
4	Canned	No	No	Yes

Train a decision tree by hand using top-down greedy induction. Use *entropy* (with natural log) as the impurity measure. Since the data can be classified without error, the stopping criterion will be no impurity in the leaves.

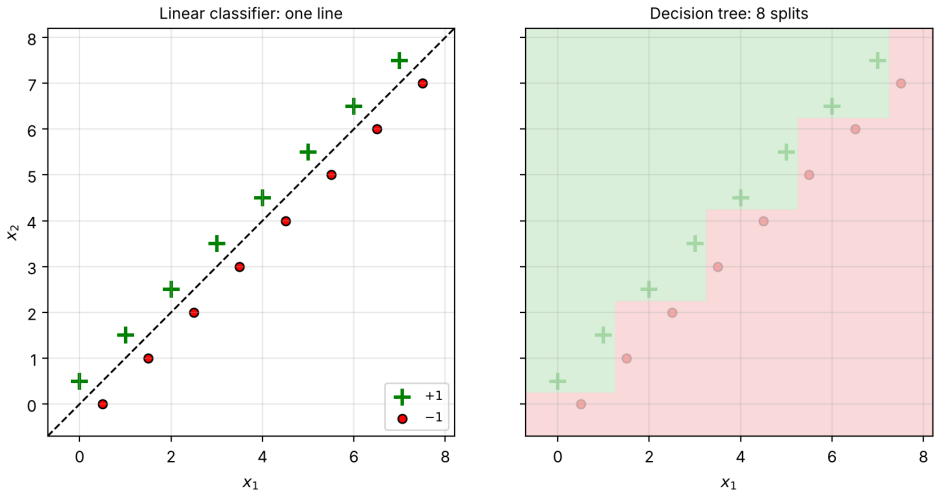
Submit a drawing of your tree showing the impurity reduction yielded by each split (including root) in your decision tree.

Solution A: At the root, $H = -\frac{1}{4}\ln\frac{1}{4} - \frac{3}{4}\ln\frac{3}{4} \approx 0.562$. Each of the three attributes sends one pure pair to one side and a 50/50 pair to the other, so the weighted child entropy is $\frac{1}{2}\ln 2 \approx 0.347$ and every root split reduces entropy by $0.562 - 0.347 = 0.216$. This is a three-way tie, so I split on Package Type. The Bagged branch $\{2, 3\}$ is already pure. The Canned branch $\{1, 4\}$ has $H = \ln 2 \approx 0.693$; splitting on fat (price works equally well) gives two pure leaves, a reduction of 0.693 (or 0.347 counted over the whole dataset). The two reductions add up to $0.216 + 0.347 = 0.562 = H_{\text{root}}$, so no impurity is left.

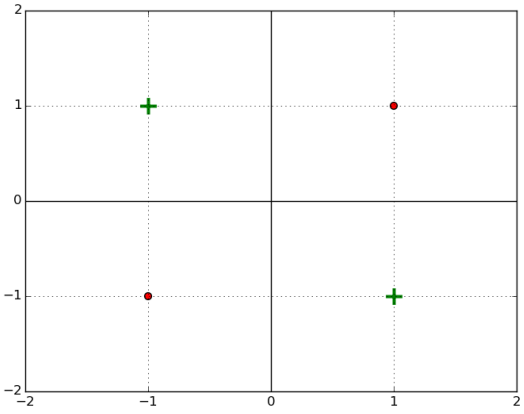


Problem B [4 points]: Compared to a linear classifier, is a decision tree always preferred for classification problems? If not, draw a simple 2-D dataset that can be perfectly classified by a simple linear classifier but which requires an overly complex decision tree to perfectly classify.

Solution B: No. A tree can only make axis-aligned cuts, so when the true boundary is oblique it has to approximate it with a staircase. In the figure, one line ($x_2 = x_1$) separates all 16 points, while the tree needs 8 splits, and the number of splits grows with the number of points. Such a tree is also hard to read and sensitive to small changes in the data, so it overfits more easily than the linear model.



Problem C [15 points]: Consider the following 2D data set:



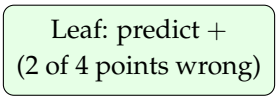
i. [5 points]: Suppose we train a decision tree on this dataset using top-down greedy induction, with the Gini index as the impurity measure. We define our stopping condition to be if no split of a node results in any reduction in impurity. Submit a drawing of the resulting tree. What is its classification error ((number of misclassified points) / (number of total points))?

ii. [5 points]: Submit a drawing of a two-level decision tree that classifies the above dataset with zero classification error. (You don't need to use any particular training algorithm to produce the tree.)

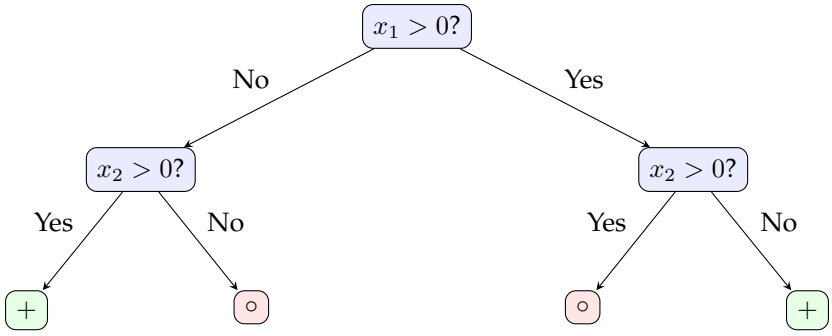
Is there any impurity measure (i.e. any function that maps the data points under a particular node in a tree to a real number) that would have led top-down greedy induction with the same stopping condition to produce the tree you drew? If so, give an example of one, and briefly describe its pros and cons as an impurity measure for training decision trees in general (on arbitrary datasets).

iii. [5 points]: Suppose there are 100 data points in some 2-D dataset. What is the largest number of unique thresholds (i.e., internal nodes) you might need in order to achieve zero classification training error (on the training set)? Please justify your answer.

Solution C: i. The root has Gini $1-(\frac{1}{2})^2-(\frac{1}{2})^2=\frac{1}{2}$. Any threshold on x_1 or x_2 that splits the data puts exactly one $+$ and one $\circ$ on each side, so both children also have Gini $\frac{1}{2}$ and the weighted impurity does not change. (Thresholds outside $[-1,1]$ leave one child empty.) No split reduces the impurity, so training stops at the root: the tree is a single leaf, which predicts one class for every point. Its classification error is $2/4=50\%$.



ii. Split on $x_1>0$ at the root and on $x_2>0$ in both children; all four points are classified correctly.



Entropy, Gini and misclassification error are all functions of the class proportions in the children. Here the first split leaves every child at 50/50, so none of them can see any progress. A measure that works must look beyond the current proportions. Example: $I(S)=$ the lowest classification error that any

single split (stump) can reach on S. Then $I(\text{root}) = \frac{1}{2}$ (every stump makes 2 errors out of 4), while after splitting on x_1 each child can be split perfectly on x_2, so $I = 0$ for both. The reduction is positive, greedy induction takes this split, and the children are pure, giving exactly the tree above.

Pros: it detects feature interactions such as XOR that one-step impurity measures miss. *Cons:* every candidate split now needs its own search over stumps, which multiplies the training cost by roughly ND; it still looks only one step ahead (parity of three features would fool it); and, being based on $0/1$ error, it is flat in many situations, like the misclassification rate.

iii. The answer is 99. *Upper bound:* assume no two identical points have different labels (otherwise zero error is impossible). Any node with at least two distinct points can be split by a threshold on a coordinate where they differ, so we can keep splitting until every leaf holds a single point. That gives at most 100 leaves, and a binary tree with L leaves has $L-1$ internal nodes, so at most 99 thresholds. *This is attained:* put the points at (k, k), $k = 1, \ldots, 100$, with alternating labels. A leaf is an axis-aligned box, and a box containing (a, a) and (b, b) also contains every (k, k) between them, hence two neighbours with different labels. So every leaf may hold at most one point, which requires 100 leaves and 99 thresholds.

Problem D [4 points]: Suppose in top-down greedy induction we want to split a leaf node that contains N data points composed of D continuous features. What is the worst-case complexity (big-O in terms of N and D) of the number of possible splits we must consider in order to find the one that most reduces impurity? Please justify your answer.

Note: Recall that at each node-splitting step in training a DT, you must consider all possible splits that you can make. While there are an infinite number of possible decision boundaries since we are using continuous features, there are not an infinite number of boundaries that result in unique child sets (which is what we mean by "split").

Solution D: $O(ND)$. For one continuous feature, sort the N values. Two thresholds give the same split whenever they fall between the same pair of consecutive values, so there are at most $N - 1$ distinct splits per feature. Over D features that is at most $D(N - 1) = O(ND)$ splits. (If we sort once per feature and update the class counts as the threshold sweeps, each split is scored in $O(1)$, so finding the best one costs $O(DN \log N)$ in total.)

2 Overfitting Decision Trees [30 Points]

Relevant materials: Lecture 5

In this problem, you will use the Diabetic Retinopathy Debrecen Data Set, which contains features extracted from images to determine whether or not the images contain signs of diabetic retinopathy. Additional information about this dataset can be found at the link below:

https://archive.ics.uci.edu/ml/datasets/Diabetic+Retinopathy+Debrecen+Data+Set

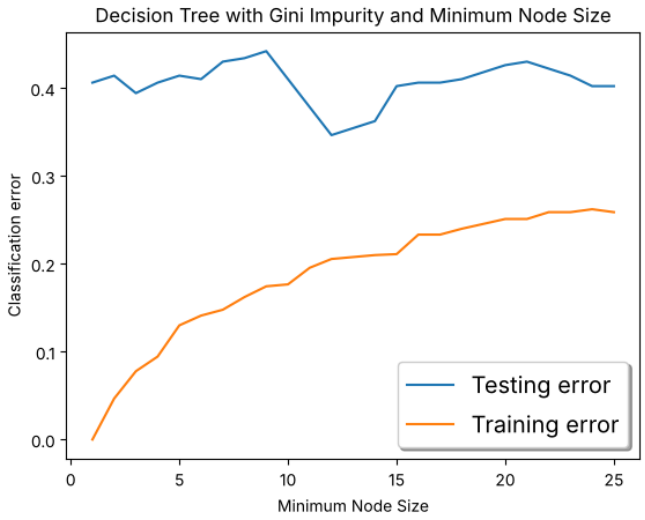
In the following question, your goal is to predict the diagnosis of diabetic retinopathy, which is the final column in the data matrix. Use the first 900 rows as training data, and the last 251 rows as validation data. Please feel free to use additional packages such as Scikit-Learn. Include your code in your submission.

Problem A [7 points]: Train a decision tree classifier using Gini as the impurity measure and minimal leaf node size as early stopping criterion. Try different minimal leaf node sizes from 1 to 25 in increments of 1. Then, on a single plot, plot both training and test classification error versus leaf node size. To do this, fill in the `classification_err` and `eval_tree_based_model_min_samples` functions in the code template for this problem.

Solution A: Code (the notebook is based on the provided template): https://colab.research.google.com/drive/PASTE_YOUR_PROB2_LINK

```
def classification_err(y, real_y):
    err = np.mean(np.asarray(y) != np.asarray(real_y))
    return float(err)

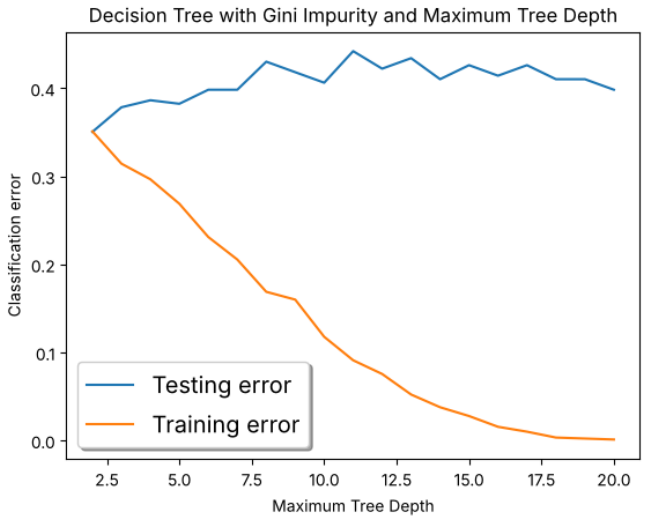
def eval_tree_based_model_min_samples(clf, min_samples_leaf, X_train, y_train, X_test, y_test
    ):
    train_err, test_err = [], []
    for m in min_samples_leaf:
        clf.set_params(min_samples_leaf=int(m))
        clf.fit(X_train, y_train)
        train_err.append(classification_err(clf.predict(X_train), y_train))
        test_err.append(classification_err(clf.predict(X_test), y_test))
    return np.array(train_err), np.array(test_err)
```



Problem B [7 points]: Train a decision tree classifier using Gini as the impurity measure and maximal tree depth as early stopping criterion. Try different tree depths from 2 to 20 in increments of 1. Then, on a single plot, plot both training and test classification error versus tree depth. To do this, fill in the `eval_tree_based_model_max_depth` function in the code template for this problem.

Solution B:

```
def eval_tree_based_model_max_depth(clf, max_depth, X_train, y_train, X_test, y_test):
    train_err, test_err = [], []
    for d in max_depth:
        clf.set_params(max_depth=int(d))
        clf.fit(X_train, y_train)
        train_err.append(classification_err(clf.predict(X_train), y_train))
        test_err.append(classification_err(clf.predict(X_test), y_test))
    return np.array(train_err), np.array(test_err)
```



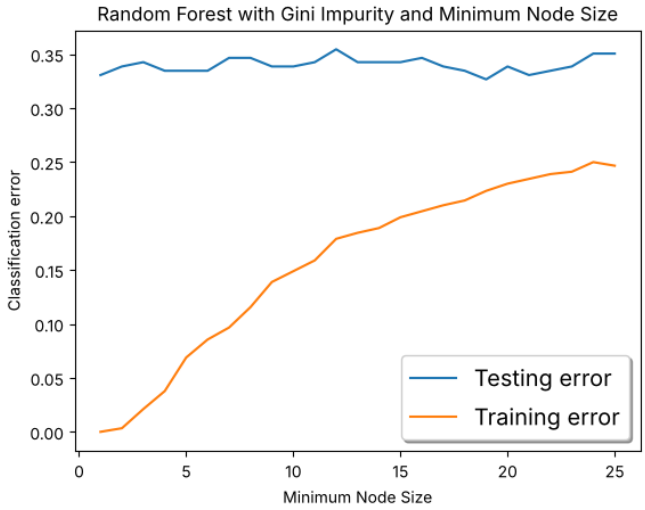
Problem C [4 points]: For both the minimal leaf node size and maximum depth parameters tested in the last two questions, which parameter value minimizes the test error? What effects does early stopping have on the performance of a decision tree model? Please justify your answer based on the two plots you derived.

Solution C: The test error is lowest at a minimum leaf size of **12** (test error 0.347) and at a maximum depth of **2** (test error 0.351).

Early stopping limits how closely the tree can fit the training set. Training error rises as the constraint gets tighter (from 0 at leaf size 1 to about 0.26 at 25), and it falls to almost 0 as depth grows to 20. Test error moves the other way: the almost fully grown trees (leaf size 1: 0.406; depth 20: 0.398) memorize the training set but generalize worse than the restricted ones (0.347 and 0.351). For depth, the test error mostly rises as the tree gets deeper, which is the classic overfitting pattern. The leaf-size curve is noisier and only dips in the 10–14 range. Since the test set has just 251 points (one point is 0.4%), I would trust the overall trend more than the exact argmin.

Problem D [2 points]: Train a random forest classifier using Gini as the impurity measure, minimal leaf node size as early stopping criterion, and 1,000 trees in the forest. Try different node sizes from 1 to 25 in increments of 1. Then, on a single plot, plot both training and test classification error versus leaf node size.

Solution D:



Problem E [2 points]: Train a random forest classifier using Gini as the impurity measure, maximal tree depth as early stopping criterion, and 1,000 trees in the forest. Try different tree depths from 2 to 20 in increments of 1. Then, on a single plot, plot both training and test classification error versus tree depth.

Solution E:



Problem F [4 points]: For both the minimal leaf node size and maximum depth parameters tested, which parameter value minimizes the random forest test error? What effects does early stopping have on the performance of a random forest model? Please justify your answer based on the two plots you derived.

Solution F: The random forest's test error is lowest at a minimum leaf size of **19** (0.327) and a maximum depth of **18** (0.315). Both curves are very flat, though: the test error stays between about 0.31 and 0.37 for every setting, and the best and typical values differ by only a few of the 251 test points.

So early stopping has little effect on a random forest. The training error behaves as for a single tree (it reaches 0 for leaf size 1 and for depth ≥ 13), but the test error does not get worse when the trees are grown fully; if anything, deeper trees are slightly better (depth 2: 0.359 vs. depth 18: 0.315). Unlike a single tree, a forest does not need early stopping to avoid overfitting.

Problem G [4 points]: Do you observe any differences between the curves for the random forest and decision tree plots? If so, explain what could account for these differences.

Solution G: Yes. (1) The forest's test error is lower than the single tree's (≈ 0.31–0.35 vs. 0.35–0.44) and its curves are much flatter and smoother. A single tree's test error goes up as it gets more complex and jumps around from one setting to the next. (2) The reason is variance. A fully grown tree has low bias but high variance: a small change in the training set gives a different tree. A random forest averages 1000 trees trained on bootstrap samples with random feature subsets, so their individual errors are partly decorrelated and the average has much lower variance at about the same bias. This is why making the trees more complex hurts a single tree but not the forest, and why the forest does not need early stopping.

3 The AdaBoost Algorithm [40 points]

Relevant materials: Lecture 6

In this problem, you will show that the choice of the α_t parameter in the AdaBoost algorithm corresponds to greedily minimizing an exponential upper bound on the loss term at each iteration.

Problem A [3 points]: Let $h_t : \mathbb{R}^m \to \{-1, 1\}$ be the weak classifier obtained at step t, and let α_t be its weight. Recall that the final classifier is

$$H(x) = \text{sign}(f(x)) = \text{sign}\left(\sum_{i=1}^{T} \alpha_t h_t(x)\right).$$

Suppose $\{(x_1, y_1), ..., (x_N, y_N)\} \subset \mathbb{R}^m \times \{-1, 1\}$ is our training dataset. Show that the training set error of the final classifier can be bounded from above if an an exponential loss function is used:

$$E = \frac{1}{N}\sum_{i=1}^{N} \exp(-y_i f(x_i)) \geq \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}(H(x_i) \neq y_i),$$

where $\mathbb{1}$ is the indicator function.

Solution A: Fix i. If $H(x_i) = y_i$, the indicator is 0 and $\exp(-y_i f(x_i)) > 0$. If $H(x_i) \neq y_i$, then $\text{sign}(f(x_i)) \neq y_i$, so $y_i f(x_i) \leq 0$ and $\exp(-y_i f(x_i)) \geq 1 = \mathbb{1}(H(x_i) \neq y_i)$. Either way $\exp(-y_i f(x_i)) \geq \mathbb{1}(H(x_i) \neq y_i)$. Averaging over $i = 1, \ldots, N$ gives $E \geq \frac{1}{N}\sum_i \mathbb{1}(H(x_i) \neq y_i)$.

Problem B [3 points]: Find $D_{T+1}(i)$ in terms of Z_t, α_t, x_i, y_i, and the classifier h_t, where T is the last timestep and $t \in \{1, \ldots, T\}$. Recall that Z_t is the normalization factor for distribution D_{t+1}:

$$Z_t = \sum_{i=1}^{N} D_t(i) \exp(-\alpha_t y_i h_t(x_i)).$$

Solution B: By definition $D_{t+1}(i) = \dfrac{D_t(i)\, e^{-\alpha_t y_i h_t(x_i)}}{Z_t}$ and $D_1(i) = \frac{1}{N}$. Unrolling the recursion,

$$D_{T+1}(i) = \frac{1}{N}\prod_{t=1}^{T} \frac{e^{-\alpha_t y_i h_t(x_i)}}{Z_t} = \frac{\exp\big(-y_i \sum_{t=1}^{T} \alpha_t h_t(x_i)\big)}{N \prod_{t=1}^{T} Z_t}.$$

Problem C [2 points]: Show that $E = \sum_{i=1}^{N} \frac{1}{N} e^{\sum_{t=1}^{T} -\alpha_t y_i h_t(x_i)}$.

Solution C: Since $f(x_i) = \sum_{t=1}^{T} \alpha_t h_t(x_i)$, we have $\exp(-y_i f(x_i)) = \exp\left(\sum_{t=1}^{T} -\alpha_t y_i h_t(x_i)\right)$. Substituting this into the definition of E gives $E = \sum_{i=1}^{N} \frac{1}{N} e^{\sum_{t=1}^{T} -\alpha_t y_i h_t(x_i)}$.

Problem D [5 points]: Show that

$$E = \prod_{t=1}^{T} Z_t.$$

Hint: *Recall that $\sum_{i=1}^{N} D_t(i) = 1$ because D is a distribution.*

Solution D: D_{T+1} is a distribution, so by Problem B and Problem C

$$1 = \sum_{i=1}^{N} D_{T+1}(i) = \frac{1}{\prod_{t=1}^{T} Z_t} \sum_{i=1}^{N} \frac{1}{N} e^{\sum_{t=1}^{T} -\alpha_t y_i h_t(x_i)} = \frac{E}{\prod_{t=1}^{T} Z_t}.$$

Hence $E = \prod_{t=1}^{T} Z_t$.

Problem E [5 points]: Show that the normalizer Z_t can be written as

$$Z_t = (1 - \epsilon_t) \exp(-\alpha_t) + \epsilon_t \exp(\alpha_t)$$

where ϵ_t is the training set error of weak classifier h_t for the weighted dataset:

$$\epsilon_t = \sum_{i=1}^{N} D_t(i) \mathbb{1}(h_t(x_i) \neq y_i).$$

Solution E: Since $y_i, h_t(x_i) \in \{-1, 1\}$, $y_i h_t(x_i) = 1$ if h_t classifies x_i correctly and -1 otherwise. Split the sum in Z_t accordingly:

$$Z_t = \sum_{i:\, h_t(x_i) = y_i} D_t(i) e^{-\alpha_t} + \sum_{i:\, h_t(x_i) \neq y_i} D_t(i) e^{\alpha_t} = (1 - \epsilon_t) e^{-\alpha_t} + \epsilon_t e^{\alpha_t},$$

because the weight of the misclassified points is ϵ_t and, as D_t sums to 1, the weight of the correct ones is $1 - \epsilon_t$.

Problem F [2 points]: We derived all of this because it is hard to directly minimize the training set error, but we can greedily minimize the upper bound E on this error. Show that choosing α_t greedily to minimize Z_t at each iteration leads to the choices in AdaBoost:

$$\alpha_t^* = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right).$$

Solution F: Assume $0 < \epsilon_t < 1$. Differentiating $Z_t(\alpha) = (1-\epsilon_t)e^{-\alpha} + \epsilon_t e^{\alpha}$ gives $Z_t'(\alpha) = -(1-\epsilon_t)e^{-\alpha} + \epsilon_t e^{\alpha}$. Setting this to 0:

$$e^{2\alpha} = \frac{1-\epsilon_t}{\epsilon_t} \quad \Longrightarrow \quad \alpha_t^* = \frac{1}{2}\ln\frac{1-\epsilon_t}{\epsilon_t}.$$

Since $Z_t''(\alpha) = Z_t(\alpha) > 0$, Z_t is convex and this stationary point is the global minimum.

Problem G [14 points]: Implement the `GradientBoosting.fit()` and `AdaBoost.fit()` methods in the notebook provided for you. Some important notes and guidelines follow:

- For both methods, make sure to work with the class attributes provided to you. Namely, after `GradientBoosting.fit()` is called, `self.clfs` should be appropriately filled with the `self.n_clfs` trained weak hypotheses. Similarly, after `AdaBoost.fit()` is called, `self.clfs` and `self.coefs` should be appropriately filled with the `self.n_clfs` trained weak hypotheses and their coefficients, respectively.
- `AdaBoost.fit()` should additionally return an (N, T) shaped numpy array `D` such that `D[:, t]` contains D_{t+1} for each $t \in \{0, \ldots,$ `self.n_clfs`$\}$.
- For the `AdaBoost.fit()` method, **use the 0/1 loss** instead of the exponential loss.
- The only Sklearn classes that you may use in implementing your boosting fit functions are the DecisionTreeRegressor and DecisionTreeClassifier, not GradientBoostingRegressor.

Solution G: Code (the notebook is based on the provided template): https://colab.research.google.com/drive/PASTE_YOUR_PROB3_LINK

```
# GradientBoosting.fit: each tree is fit to the current residual
self.clfs = []
residual = np.array(Y, dtype=float)
for _ in range(self.n_clfs):
    clf = DecisionTreeRegressor(max_leaf_nodes=n_nodes)
    clf.fit(X, residual)
    residual = residual - clf.predict(X)
    self.clfs.append(clf)

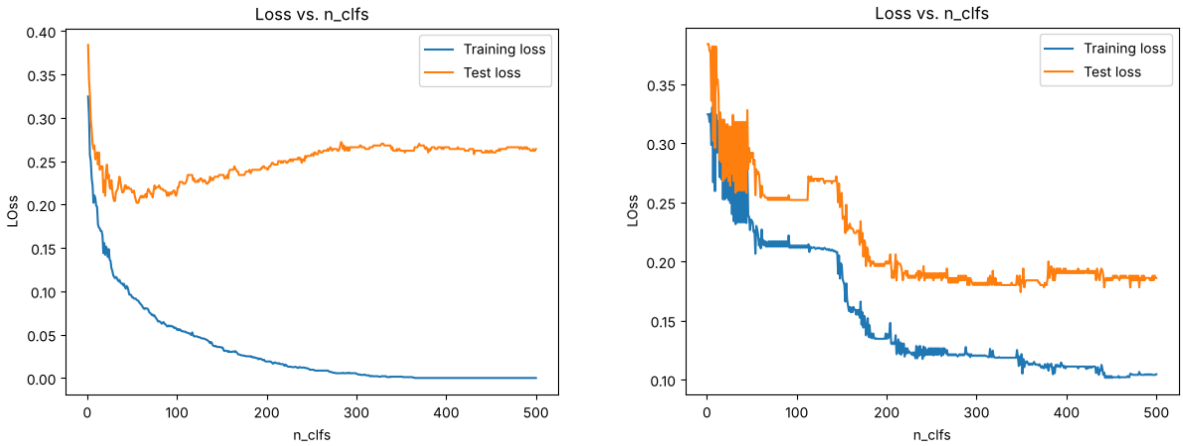
# AdaBoost.fit
N = len(X)
self.clfs, self.coefs = [], []
D = np.zeros((N, self.n_clfs))   # D[:, t] holds D_{t+1}
w = np.ones(N) / N
for t in range(self.n_clfs):
    clf = DecisionTreeClassifier(max_leaf_nodes=n_nodes)
    clf.fit(X, Y, sample_weight=w)
    pred = clf.predict(X)
    eps = np.sum(w * (pred != Y))                # weighted 0/1 error
    eps = np.clip(eps, 1e-10, 1 - 1e-10)
```

```
    alpha = 0.5 * np.log((1 - eps) / eps)
    w = w * np.exp(-alpha * Y * pred)
    w = w / np.sum(w)
    self.clfs.append(clf)
    self.coefs.append(alpha)
    D[:, t] = w
return D
```

With 500 weak learners the results are: gradient boosting has training loss 0.000 and test loss 0.264; AdaBoost has training loss 0.1045 and test loss 0.186.

Problem H [2 points]: Describe and explain the behaviour of the loss curves for gradient boosting and for AdaBoost. You should consider two kinds of behaviours: the smoothness of the curves and the final values that the curves approach.

Solution H:



Smoothness. Gradient boosting (left) is smooth: each regressor is fit to the residual, so every step makes a small change to the ensemble and the training loss falls almost monotonically. AdaBoost (right) is jagged and step-like, with the loss going up on many iterations. The $0/1$ loss is not what AdaBoost optimizes (it greedily reduces the exponential bound), and one new learner with a large coefficient can flip many points at once.

Final values. Gradient boosting reaches training loss 0 (after about 365 regressors) because it keeps fitting the residuals, noise included. Its test loss is best around 0.20 (about 55 regressors) and then creeps up to 0.264, a sign of overfitting. AdaBoost levels off at training loss ≈ 0.105 and test loss ≈ 0.186 and does not overfit in this range. Later learners face weights concentrated on the noisy overlap region, where they do barely better than chance ($\epsilon_t \approx 0.497$, $\alpha_t \approx 0.006$), so they hardly change

the ensemble.

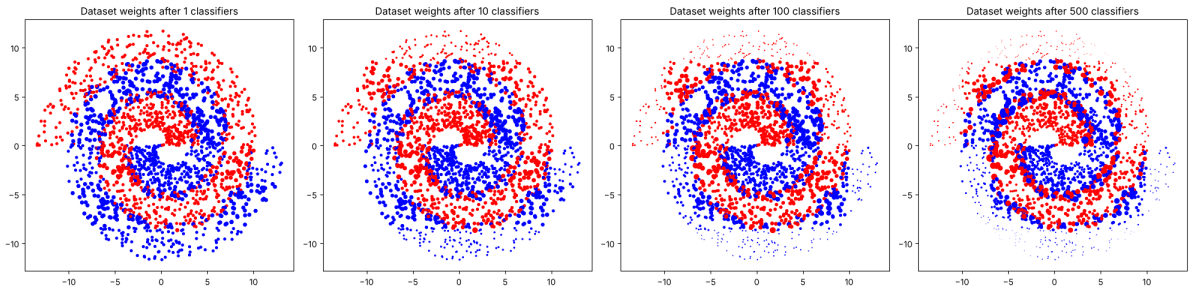
Problem I [2 points]: Compare the final loss values of the two models. Which performed better on the classification dataset?

Solution I: Gradient boosting has the lower training loss (0 vs. 0.105), but AdaBoost has the lower test loss (0.186 vs. 0.264). Since test loss measures generalization, AdaBoost performed better on this classification dataset.

Problem J [2 points]: For AdaBoost, where are the dataset weights the largest, and where are they the smallest?

Hint: *Watch how the dataset weights change across time in the animation.*

Solution J:



The weights are largest for points in the band where the two spirals overlap, i.e. near the decision boundary and among the noisy points, which are repeatedly misclassified. For example, the final weight of the training points the ensemble gets wrong averages 1.5×10^{-3}, vs. 3.8×10^{-4} for points it gets right. The weights are smallest for points far from the boundary, such as the outer ends of the spiral arms (radius above 12), which are classified correctly from the first few rounds and keep being down-weighted. Their weights shrink exponentially, down to about 10^{-7}.